

Team Costco Hotdog Future Engineers 2026 Engineering Journal

Table of Contents

1. Introduction.....	2
2. Design Goals.....	3
3. Mechanical design.....	7
4.1 Wiring & Electrical Architecture.....	14
5. Sensors & Perception.....	16
6. Autonomous Driving Algorithm.....	19
7. Control System.....	21
8. Obstacle Avoidance Strategy.....	25
9. Parking Strategy.....	26
10. Testing & Experiments.....	26
11. Design Iterations.....	33
12. Problems & Solutions.....	35
13. Performance Analysis.....	36
14. Final Robot.....	37
15. Reproducibility.....	40
16. GitHub / Code.....	44
17. Engineering Goals.....	47
18. Future Improvements.....	47
19. Links.....	49

1. Introduction

We are team Costco Hotdog, participating in the Future Engineers category of WRO 2026, and from the Explorer Robotics club based in Canada.

This document is designed to showcase our decisions, tests, failures, iterations, reasoning for each component, robot reproducibility, and the main process we took to engineer our robot for the competition.

1.1 Meet the team

Our team, Costco Hotdog is based off of a infamous \$1.50 CAD hotdog in our country. We are composed of two members and our coach. Our group is based out of the Durham region in Ontario, Canada.

The two members of our team are Jonathan Huang and Walter Wu. We are both going into our second year of high school. Walter is enrolled in the AMP for All Saints studying media arts and Jonathan is attending Notre Dame and likes to learn about the origins of aerospace mechanisms and discovering how they work. Jonathan is the main programmer for all the software and helps a lot with any mechanical or hardware problems fixing most of the problems as soon as they arise. Walter is the main person behind the documentation and hardware and mechanical engineering.

1.2 Challenge

WRO Future Engineers is a competition where teams must design, build, and program a fully autonomous driving robot that completes two challenges. The robot by itself will navigate a track, following any random sections while avoiding obstacles and staying within the track boundaries. The track itself can include obstacles, a parallel parking challenge, and changing track lengths. The future engineers category is split up into three main challenges, the open challenge, obstacle challenge, and documentation.

The main challenge is making the robot reliable enough to handle changes in the track while moving quickly and accurately. Any mistake on the steering mechanism, the sensors or programming may lead to our robot colliding with the walls or failing to detect objects and any other uncertainties that come with the problem greatly influence the way it is solved. The objective of our team in solving the problem is to have a robot which will be able to navigate the path and complete the circuit without any faults.

2. Design Goals

The Future Engineers rulebooks provides many well rules that influenced our designing for the robot, those include:

- Robot dimensions
- Motor limitations
- Wired communication requirements
- Track requirements
- Starting Requirements
- Sensors/cameras
- Power requirements
- Parking requirements
- Autonomous operation
- Drive System limitations

The three main rules contributing to our design goals were the parking requirements, drive system limitations, and the starting requirements:

The starting requirements are particularly strict as they indicate that the robot must begin completely switched off, only one button/switch is used to turn it on, and it must enter a waiting state until another button input to start. This is not inherently complicated but not only can it fail easily as evident in previous infractions in past competitions but when it does the consequences can severely affect the scoring.

The next problem is the parallel parking requirements, teams have an option whether or not to incorporate the parking in their run since the challenge is so competitive it is difficult to get a high score in comparison to other teams without the twenty-two points that the parking provides. The parking itself can be treated as a challenge separate to the open or obstacle challenges not only because it is optional, but because of the requirements. The parking lot is only 1.5x the length of our robot and deciding whether or not the parallel parking was successful comes down to a difference of 2cm.

The rulebook provides specific limitations surrounding the drive systems and included a note on how previous teams have received a dock in points or disqualification because they did not adhere to the rules. The drive system must be physically connected either through a gearbox or other method, and it is prohibited to use one motor for each side. This is something every team has to consider when designing their drive systems.

2.1 Brainstorming & Initial Concepts

Before we made the final design, there were other designs that we had to consider, ranging from mechanical to electrical as well as software solutions. The purpose of the brainstorming process was to come up with a design that was able to effectively navigate through the competition environment and yet was still easy to construct, reproduce and modify.

Concept A – Front Wheel Steering

Our first concept was a front wheel steering design from cars where the front wheels turn and the rear wheels provide the power.

Advantages:

- It is more like the typical car design.
- Better ability to control the direction of the robot.
- Stable when going straight.
- The steering is separate from the motor that drives the robot.

Disadvantages:

- More mechanically complex.
- Needs a steering mechanism and servo mechanism.
- Has a larger turning circle.
- Difficult to make quick corrections around objects.

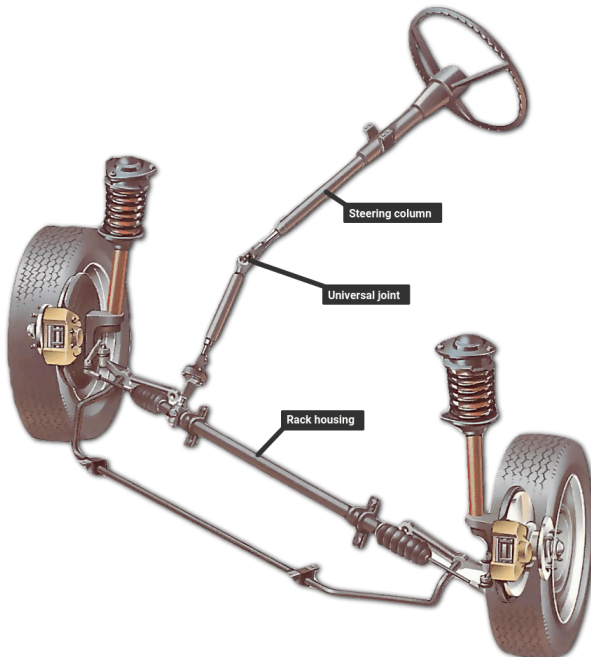


Figure 1. A picture of a real world application of the front wheel steering concept.

Concept B – Differential Drive

The second concept consisted of two independently powered wheels that would enable the robot to turn based on the different speeds and directions of the wheels on either side.

Advantages:

- Mechanically simple
- Relatively few moving parts
- Good ability to turn
- Easy control using software
- Our team was familiar with this kind of driving style from our previous competitions using EV3

Disadvantages:

- Different from conventional car steering
- Can lead to wheel slippage when turning
- Not well suited to our driving behavior requirement
- Can be difficult to position precisely on some terrains

Concept C – Front Steering with Rear-Wheel Drive

We have also considered the advantages of combining the strengths of the two methods with front wheel steering combined with a drive motor connected to the rear wheels.

Advantages:

- Better control on driving route
- Relatively simple drivetrain
- Predictable turning
- Better handling when maneuvering through obstacles

Disadvantages:

- More complex than differential drive
- Steering system has to be properly aligned
- Calibration of servo positions required

Finally, based on evaluation of the concepts, we have chosen to use front wheel steering and rear-wheel drive method due to its superior performance characteristics in our anticipated competition setting, as well as to try something new while being able to talk to people who were already familiar with this kind of drive train, i.e our teammates and coach.

Chassis and Wheel Arrangement

Other types of chassis layout were also taken into consideration. Many different wheel sizes, wheelbases, and locations of battery and electronic components were tried out. With a shorter chassis, tighter turns can be made but would become unstable at high speeds, while a longer chassis will give more stability to the chassis and space for components, but we would have to consider weight with everything. A good compromise was found with a compact layout which gave enough room for all the necessary hardware.

Sensor Placement

We had considered several locations for the sensor mix and matching the height, and distance from the front of the car. Mounting the camera at a higher point would help us capture the whole race track area whereas mounting at a lower location would help in getting consistent readings regarding the obstacles. The sensor could also have been mounted in the front area of the robot so that information regarding the obstacles would be captured well before reaching the obstacles.

The final placement was decided so as to capture the most important information, which would be the area in front of the robot; as well as the side walls.

Software Approaches

A number of navigation approaches were looked at, most importantly:

Fixed movement sequences: easy to program but unable to respond to unexpected situations or conditions.

Distance-based navigation: determines through distances when the robot needs to change direction or position itself.

Camera-based navigation: relies on the camera to find features in the course and the robot's position.

Multiple sensors: relies on a variety of information sources to enable the robot to make better decisions.

We decided to use an approach based on sensors which combines the camera data and distances measurements as well as robot control data. This allows the Raspberry Pi 5 to analyze the surroundings and make decisions regarding navigation rather than executing preprogrammed actions.

Obstacle Avoidance Strategies

We also looked at different ways of navigating around obstacles. One such method was the use of a preset path, as in moving 50cm right when the robot sees a red pillar, but this would be less effective if there were any changes in the position of the robot. The other one is the continuous detection of the obstacle and calculating the correct steering angle.

Our decision on the strategy would be that the robot detects the environment and steers according to the sensor data, continuous detection and adaption. This makes the robot flexible in case of change in position or any other affections.

Final Design Decision

After evaluating all of the different designs, the one that offered the most balance between reliability, maneuverability, sensor detection, mechanical simplicity, and control via software was found. In the final design, the robot is equipped with a small chassis, front wheel steering, rear wheel drive, Raspberry Pi 5 computer, Arduino, and several sensors that gather data from the environment.

The Brainstorming process turned out to be incredibly useful as it made it possible for us to see the weaknesses of the initial designs before proceeding to finalize the robot. Instead of using our first design concepts and changing the robot through trial and error, we evaluated several options and based on their advantages and drawbacks, came up with the final version of the robot.

3. Mechanical design

The mechanical design of the robot was completed through multiple iterations of prototyping, testing, and redesigning. Our primary objectives throughout the mechanical design process were to create a robot which was compact, steady, maneuverable, strong enough to withstand repeated testing, and capable of holding all the electrical sensors necessary for the competition.

The mechanical design kept changing as we encountered many problems through our testing. Among the major mechanical problems we encountered were those to do with wheel attachment, our undesirable center of gravity, the mounting of the BLDC motors, and the chassis design in itself. Rather than seeing this as failure and completely scratching previous ideas, we saw it as an opportunity to improve our design.

3.1 Chassis

The chassis provides the main structural frame of the robot and supports the drivetrain, steering system, battery, electronics, and sensors. We designed the chassis around the size limitations of the competition while leaving enough room for all of the required components.

Dimension	Dimension
Length	21cm
Width	10cm
Height	22.5cm

Final dimensions were taken from the complete robot rather than estimates from early CAD designs

The chassis has been made mostly out of 3D printed parts, tape, screws, brackets, and other mechanical fasteners. Thanks to 3D printing, we were able to make and redesign custom parts quickly without making each of them manually.

The first issue which we have faced is that we underestimated how much time each 3D print would take. On March 29, 2026, we recognized that printing time was a big limitation. This was fixed by scheduling even more of our prints between classes, and designing them all before the next class would start so that we would be able to get our parts printed without compromising our progress.

Another early limitation was the lack of screws for assembling the mechanism. Between March 1 and April 19, 2026, this has been happening on a regular basis. It was pretty easily resolved by buying a set of screws in different sizes, but was a notable challenge in the beginning of creating our robot.

Mounting Points

The chassis houses mounting points for:

- Furitek Micro Komodo motor
- Furitek Lizard Pro ESC
- Steering servo
- Steering linkage
- Battery
- Raspberry Pi 5
- Arduino Nano
- Camera
- D500 LiDAR
- Wheels

The specific mounting locations were chosen in such a way that essential components could be mounted securely, without changing the center of gravity too much.

Chassis Strength

The chassis had to be able to handle the stresses of driving, turning, impact, and some replacement of components. While we tested the chassis, we noticed that certain parts were able to come loose after time passed. This included the wheels and the mounting screws used for the motor.

The problem with wheel attachment began on March 7, 2026. At times, the wheels would either loosen or come right off. This proved that simply mounting the wheels was not enough for competition.

Centre of Gravity

One of the most significant mechanical problems we faced was heavy wobbling even while taking shallow turns.

On May 24, 2026, it was observed that the center of gravity of the robot was exceedingly high; and because of that, the robot becomes very unstable, especially more while taking sharp steering turns. The earlier configuration was such that the robot had its weight more towards the front and high.

The redesigning of the chassis configuration helped in addressing this problem by relocating the battery towards the back of the robot, helping with counteracting the old front-heavy design and overall improved the weight distribution of the robot.

This redesigned layout solved the issue and proved to be effective in highlighting the fact that the mass distribution of the robot should also be taken into high consideration along with its dimensions.

3.2 Drive System

The robot uses a brushless drive system consisting of a **Furitek Micro Komodo motor** and **Furitek Lizard Pro ESC**. The drive system was selected to provide enough power for rapid acceleration while maintaining controllability during autonomous navigation.

The Micro Komodo BLDC motor was chosen because it gives adequate power output and can fit into the chassis easily due to its small size. The Lizard Pro ESC has been chosen to electronically control the brushless motor and allow the Raspberry Pi/Arduino to control the drive system.

Motor Mounting Problem

On May 5, 2026, we found out that the BLDC motor could potentially come loose while running. Once the screws holding the motor in place became loose, the pinion gear could detach itself from the other gears on the drivetrain. The loss of proper engagement meant that the robot couldn't perform something as simple as driving forward.

The solution was simple, we just needed to tighten the screws keeping the motor in place.

Although the solution was relatively simple, this was an important change from a mechanical point of view, since it showed that alignment of the drivetrain is equally as important as choosing the motor. It does not matter how strong your motor is if it cannot transmit the torque properly.

Drive Performance

The maximum speed and acceleration were calculated by testing and not by simply assuming manufacturer's specifications. The robot was tested in a controlled environment, and the time needed to cover known distances was measured.

The measured results are as follows:

Maximum speed: 32.23 m/s (100% throttle)

Acceleration: 5.89 m/s²

Average speed: 4.1 – 6.8 m/s (15-25% throttle)

3.3 Steering

The robot has a steering mechanism which is controlled by the servo. The servo manipulates the mechanical linkage which alters the orientation of the front wheels.

Component	Specification
Steering servo	HiTEC HS-5055MG Digital Micro Servo
Steering mechanism	Front wheel steering
Steering Linkage	Parallelogram Linkage
Maximum left angle	42°
Maximum right angle	122°
Centre angle	82°

Steering Calibration

The theoretical center value of the servo was not considered as the inherently correct or right center value. We had tested the steering system and adjusted our software according to the physical center value of the wheels to find the actual "straight" position for the wheels.

For instance, the initially assigned command of 90 μ s to the servo was assumed to be the center value. After testing, it was found that the center value is actually closer to ~66-67 μ s.

Moreover, we also tested the maximum left and maximum right turning angles. After this, we programmed our software accordingly to ensure that the servo does not move outside the mechanically safe range.

Turn Radius

The turning radius of any robot is calculated mathematically by dividing the wheelbase of the vehicle by the sine of the steering angle.

$$(R = \text{Wheelbase} / \sin(\text{Steering Angle}))$$

For this mathematical method of calculating turn radius, we had calculated the turn radius or R to be ~17.8cm.

$$\text{Radius} = \text{Wheelbase} \div \sin(\text{Steering Angle})$$

$$\text{Radius} = 10.2 \div \sin(35)$$

$$\text{Radius} = 10.2 \div 0.57357643635$$

$$\text{Radius} = 17.7831573154$$

But to figure out if that is actually the real world case, the turn radius can also be determined physically by making a complete 180 degrees turn and dividing the distance of the two points (the beginning and end of the U-Turn) by two.

With our try at the physical method, the turning radius was measured by commanding the robot to maintain a constant steering angle and measuring the path followed by the centre of the robot.

The turn radius measured in at approximately **18.5cm**.

This measurement was important in deciding if the robot was capable of making the turns required in the strict competition course.

3.4 Mechanical Iterations

The mechanical design evolved considerably through the design process. The end product was not developed from one design; rather, a series of modifications were made based on problems that occurred during testing.

Version 1: Initial Chassis

The first chassis was designed for the basic drivetrain and steering needs. This chassis served as mounting points for our major components but had not yet been tested thoroughly.

Problem:

Through testing, we found that there were some problems with weight distribution, wheel attachment, and mounting points.

Modification:

We worked on the mounting points and wheel attachment and started moving our components around to find more comfortable places to put everything.

Version 2: 1st Variation Chassis

The first modification of our chassis improved component placement easier and overall made easy assembly of the robot. Nonetheless, there were still a lot of problems that arose when the robot was tested at increased speed levels.

Problem:

On May 24th, 2026, the robot started wobbling, particularly during turns. It was made obvious that the centre of gravity was way too high and the robot was also too front heavy.

Modification:

There was a major redesign of the main chassis where the biggest changes was that the battery was relocated to the back, and we got rid of most of the extra material that just served as extra, unneeded protection to the electrical components.

Wheel Mounting Modification

Problem:

Starting on March 7, 2026, there were repeated cases where the wheels kept becoming loose or falling off when being tested.

Modification:

We further improved upon the wheel mounting system and the securing of wheel hardware.

Result:

There was an immediate improvement in terms of reliability of the wheels during repeated fast-pace testing.

3.5 Mechanical Challenges and Lessons Learned

There were several challenges during development that influenced our final mechanical design.

Date	Problem	Impact	Solution / Lesson
03/07/2026	Wheels falling off	Interrupted testing and affected reliability	Improved wheel mounting and retention
03/29/2026	3D prints taking too long	Slowed prototyping	Managed prints between classes
03/01/2026 – 04/19/2026	Not enough screws	Delayed assembly and repairs	Purchased a screw set
05/05/2026	BLDC motor becoming loose	Pinion gear lost proper alignment	Tightened and secured motor mounting
05/24/2026	Robot wobbling during turns	Reduced stability at high speed	Redesigned chassis and moved battery rearward
07/06/2026	Multiple batteries failed	Prevented normal testing	Replaced batteries and changed supplier
07/27/2026	Robot turned into walls in difficult cases	Robot could fail to recover from certain obstacle approaches	Continued testing and adjusted navigation/steering behaviour
08/09/2026	Raspberry Pi failure	Stopped software development	Replaced/repared computer and recovered development
08/09/2026	Regulator short circuit	Electrical system became unsafe/non-functional	Rewired and re-soldered connections

These such issues have shown us that mechanical design is not only about creating a robot that will work one time. The robot should be reliable after all drives, tests, adjustments and repairs. Issues such as loose wheels, movement of motors, and improper weight distribution could be discovered only during testing because they were not visible at the initial stage of the design.

Thus, the final mechanical design is the outcome of a repeated sequence of steps: **design -> build -> test -> discover problems -> modify -> re-test.**

4.1 Wiring & Electrical Architecture

The electrical architecture of the robot is made in a way so that the robot distributes power from the main battery in a safe way to each device with appropriate voltage. This robot is powered by a 11.1V 2200 mAh 3-cell LiPo battery.

The electrical system is separated into two power rails. One rail is for the high-power drive and the other one goes straight to the regulator that, well, regulates the voltage to provide power to the lower-voltage electronics. Having separate lines gives a possibility for the motor to get power straight from the battery without causing damage to devices like Raspberry Pi, camera, LiDAR, and Arduino due to high voltage required for the BLDC motor from the battery.

Electrical Architecture

The overall electrical architecture of the robot is shown below:

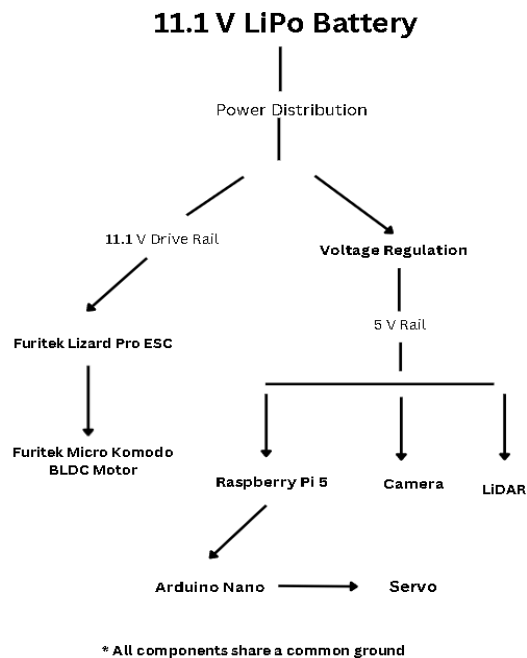


Figure 2. A hand made diagram of the electrical architecture of our robot.

4.2 Power Budget

The robot is powered by an 11.1 V 2200 mAh LiPo battery. In normal operations, the estimated average power consumption is approximately 38 W with an average current consumption of about 3.5 A. However, in situations such as high acceleration or in general high-load conditions, the maximum current drawn from the battery is approximately 15 A.

The drive motor is one of the main sources of power consumption, especially when accelerating. In addition, the Raspberry Pi 5 uses a considerable amount of power in processing the information obtained from the camera and sensors. The camera, LiDAR, Arduino, and the steering system contribute small proportions to the overall power consumption.

The power budget has been determined using a combination of both the specifications of individual components and the actual current consumption observed during the robot's operations. This is because neither the motor nor the steering system draw power consistently enough for a set in stone calculation.

Power Budget Table:

Component	Voltage (V)	Current (A)	Power (W)	Rail
Raspberry Pi 5	5.0	1.80	9.00	5V Regulated Rail
Pi Camera	5.0	0.25	1.25	5V Regulated Rail
Arduino Nano	5.0	0.03	0.15	5V Regulated Rail
D500 LiDAR	5.0	0.20	1.00	5V Regulated Rail
HiTec HS-5055MG Servo	6.0	0.15	0.90	6V BEC Rail
Furitek ESC Electronics	6.0	0.05	0.30	6V BEC Rail
Furitek Micro Komodo 1212 Motor	11.1	2.00	22.20	11.1V Direct Rail
Total Device Power			34.80 W	

There are also some minor power losses caused by the voltage regulator when converting the battery voltage to the required regulated 5V. These losses are taken into account while calculating the overall power usage of the robot.

The power system is designed to be capable of supplying the average current load as well as larger currents while accelerating or in other unexpected circumstances.

5. Sensors & Perception

The robot uses two main sensors to collect information about the environment and any changes that could come about unexpectedly, as well as making navigation decisions automatically. The two main types of sensors used for perceiving information are Raspberry Pi Camera and D500 LiDAR.

The camera is the by far most important sensor that is being used in the current navigation algorithm, which detects walls, colored pillars, and obstacles around the course.

5.1 D500 LiDAR

The D500 LiDAR measures the distance between the robot and the surrounding objects. The LiDAR sensor is placed at the very front of our robot and is oriented in such a way that the front and sides of the robot can be scanned throughout the entire course

Specifications:

Property	Value
Measurement type	DTOF
Operating range	0.5m and 2m.
Accuracy	±20 mm (For ranges between 0.5-2m)
Update rate	10Hz

LiDAR provides information about the distances of nearby objects or walls. But . The LiDAR does not distinguish colours or shapes of objects. Hence, its practicality differs from the practicality of the camera since it only provides distance information.

5.2 Raspberry Pi Camera

The Raspberry Pi Camera is the main perception sensor used by the robot's navigation system. It captures visual data allowing the robot to locate walls and spot red and green poles.

The camera is mounted at a high point on top of a stand at the top of our robot's chassis. and angled at approximately 7° to provide a clear view of the area in front of the robot.

Specifications:

Property	Value
Resolution	640 × 480 pixels
Mounting angle	$\sim 7^\circ$
Maximum frame rate	30 FPS

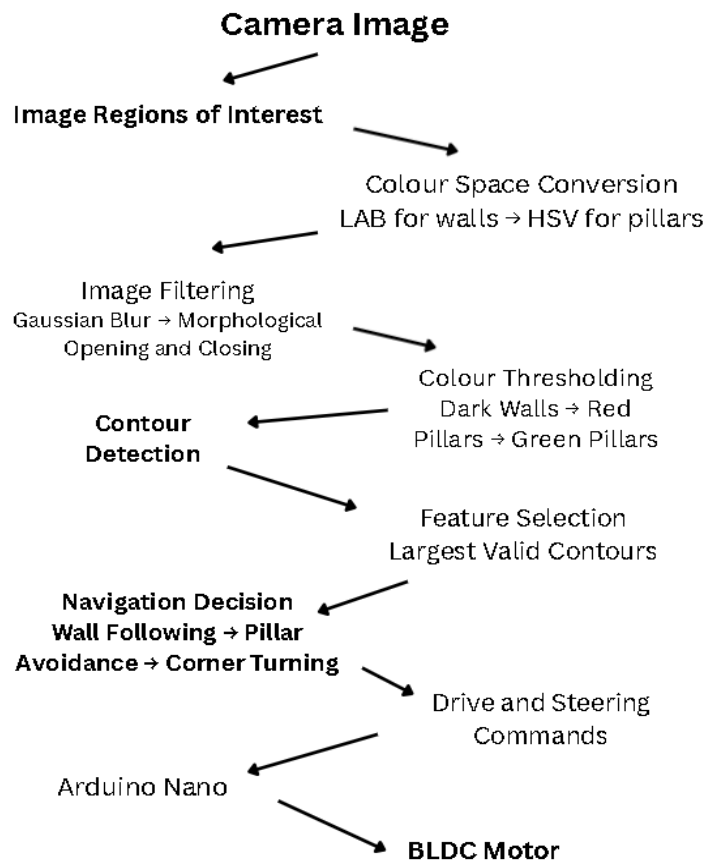
The Raspberry Pi processes each camera frame using computer vision. Different regions of the image are used for wall and pillar detection.

In order to detect walls, the image is converted to the LAB colour space and a Gaussian blur along with morphological filtering are used prior to detecting contours. The two largest valid contours on the left and right sides of the image are used to determine how much wall there is.

In order to detect pillars, the image is converted to the HSV color space and thresholds are used to find red and green pillars while ignoring smaller contours. The position of the pillar is used to determine a steering correction.

The processed data is used by the navigation system to toggle between wall following, pillar avoidance and corner turning.

Image-Processing Pipeline



Filtering

In order to minimize noise and reduce the number of wrong detections, the imaging system uses several filters. The Gaussian blur filter removes small changes in the picture, whereas morphological opening and closing get rid of noise and close holes in the contours.

When the contours are detected, the algorithm ignores those that are smaller than minimum area thresholds. In addition to this, the navigation system should have the corner detection condition be true for several consecutive frames to enter the corner turning phase.

Limitations

The camera can be heavily influenced by factors such as lighting, reflection, motion blur, and changes in the colour of objects. Image processing is also considerably more computationally intensive than distance calculations.

5.3 Sensor Placement

The sensor placement was made while keeping in mind to make sure both sensors will have a clear view of the environment of the course track and most importantly remain mounted safely.

The camera was placed to give a front view of the track including walls and pillars required by the navigation system. The LiDAR was placed in such a way that it gives distances without any obstruction from the robot's chassis.

The final positions for the sensors were chosen to maximize useful viewing area and minimize obstructions from other parts of the robot, the part we struggled most with was definitely the camera mount, as there were a multitude of different parts that could go wrong.

5.4 Combined Sensor Usage

The camera and LiDAR collect different types of information:

- Raspberry Pi Camera: Detects visual elements like walls and coloured pillars.
- D500 LiDAR: Collects the distance from the objects around.

At the moment, the software is designed in a way that allows using data collected by the camera for autonomous wall following and obstacle avoidance. The LiDAR can provide additional distance information, but is mainly used for parallel parking.

Using more than one sensor gives the robot an opportunity to have different kinds of information about the environment, and creates opportunities to improve consistency and reliability by combining visual and technical distance data.

6. Autonomous Driving Algorithm

The autonomous driving algorithm translates sensor data into instructions for the robot's steering and motors. The Raspberry Pi 5 processes camera images in real time, recognizes key objects in the competition track, and determines the robot's current driving behavior.

The navigation system can be summarized as this:

1. **Camera Input**
2. **What does the robot see?**
Walls, coloured pillars, and changes in the course
3. **Feature Detection**
Wall detection and pillar detection
4. **What should the robot do?**
Wall Following → Pillar Avoidance → Corner Turning
5. **Calculate Steering and Speed**
6. **Send Commands to Arduino Nano**
7. **Control Motor and Steering**

6.1 Inputs

The main input of the autonomous driving algorithm is the Raspberry Pi camera.

The input data from the camera includes images that are processed to then detect:

- Left and right walls
- Red pillars
- Green pillars
- Changes in visible wall area that may indicate a corner

The navigation program also takes into account the state in which it drives. This way, the robot will not try to execute more than one behavior simultaneously.

6.2 Processing and Feature Detection

The image captured from the camera is divided into regions of interest for walls and pillars.

In the wall region processing, the robot evaluates the percentage of wall seen by the camera on each side of the screen. The difference between the two sides is calculated and a steering correction then made. Pillars are identified using color thresholds where the program can identify objects that are red and green in color. It identifies their positions on the camera image and uses them to change direction to avoid the obstacle.

It is also able to detect corners through wall detection. To avoid false detection of corners, the robot must maintain multiple framers of the corner before changing its state to the corner-turning one.

6.3 Driving States

The robot employs different states for driving based on its detection.

Wall Following:

During regular driving, the robot takes into consideration the information obtained from the wall detection from the left side and the right side of the camera frame. The steering correction is applied in order to ensure that the robot is in a proper position relative to the walls.

Pillar Avoidance:

If the robot detects a red or green pillar that is considered to be a valid "target", it will temporarily switch to the obstacle avoidance mode. Depending on the position and color of the pillar, the robot calculates the direction and value of the steering.

Corner Turning:

In case the robot determines that the corner is approaching, it enters the corner turning state. The corner turn will only be made if the detection condition is stable for several consecutive frames.

Recovery Mode:

After passing a pillar, to get back to normal wall follow the robot uses the last known pillar position and wall measurements to steer back toward a stable driving position before continuing normally.

6.4 Outputs

After processing the data captured by the cameras, the Raspberry Pi identifies what outputs need to be sent.

The three main outputs are:

Steering command: Controls the position of the steering servo.

Motor command: Controls the speed at which the robot drives.

Driving state: Indicates whether the robot is in wall tracking, pillar avoidance, or corner turning mode.

These outputs are passed on to the Arduino Nano, which is responsible for low level control of the steering and drive components. The algorithm repeatedly does this while the robot operates, enabling the robot to respond to changes in its environment rather than follow a pre-programmed set of steps.

6.5 Algorithm Design

Our autonomous driving algorithm uses a state based design. The robot does not rely on one form of steering control in all situations. The robot adjusts its behaviour according to the characteristics detected by the camera.

The use of the state-based design ensures that obstacle avoidance and corner turns can become the priority momentarily during the drive rather than the wall following. After completion of the task, the robot can revert back to the normal navigation mode.

The detecting → processing → deciding → controlling cycle is continuously done by the robot to complete the course.

7. Control System

The control system takes the navigation system's decisions and translates them into actions by controlling the movements of the servo and dc motor. Although the algorithm for autonomous navigation determines what the robot should do, it is the control system that decides how to do it.

The control system can be summarized as:

1. Navigation Decision
2. Calculate Steering and Motor Command
3. Send Command to Arduino Nano
4. Control Steering Servo and Drive System
5. Robot Movement

7.1 Steering Control

The Raspberry Pi determines a steering correction using the information captured by the camera. In wall following, the difference between the left and right wall areas gives the direction in which the robot should turn either left or right.

During pillar avoidance and corner turning, the steering commands are determined according to the driving state of the robot.

The last stage involves sending the steering command to the Arduino Nano to control the steering servo.

The steering system was calibrated by setting different commands for the servo while comparing them to the physical orientation of the front wheels. The centre command was set according to the physical straightness of the wheels and not just the theoretical centre value.

7.2 Motor and Speed Control

The Arduino Nano also gets the driving commands from the Raspberry Pi and manages the Furitek Lizard Pro ESC and Micro Komodo motor.

The motor speed is chosen depending on the navigation behavior at the moment. The robot can apply various driving commands when navigating, avoiding obstacles, and turning corners.

This motor management system provides an opportunity for the autonomous algorithm to slow down the movement of the robot if another behavior of driving is required.

7.3 Feedback and Control Accuracy

The existing control system relies mostly on data from the camera in terms of feedback related to the robot's positioning relative to the surroundings.

For instance, after applying a steering correction, the new camera frames contain new information about the detected walls and obstacles and, thus, a new correction needs to be calculated, if required.

This creates a continuous control process:

Detect Environment → Calculate Error → Adjust Steering → Move → Detect Environment Again

7.4 Calibration

Calibration was essential for making sure the commands from software created the intended motion.

Tests for the steering servo were done to determine:

- The physical centre position
- Maximum safe left position
- Maximum safe right position

These values were used in the software so that all steering commands fall within the mechanical limits of the robot.

Also, motor commands were tested for suitable speeds in driving conditions. Faster speeds meant quicker completion of the course but made steering less reliable and obstacle avoidance tough.

7.5 Calculations Vs. Reality

Most of the calculations we had for the values of the servo commands differed immensely to the real life, final values. For the servo it ranges 0-180°, and 500-2500µs for the standard version, meanwhile the BLDC motor's final values didn't change at all to the calculated values, the BLDC motor's standard ESC Pulse-Width ranges from 1000-2000µs.

Servo

The servo controls the degree of rotation for our wheels with a rotation of the shaft, this shaft is usually able to reach 0-180° resulting in a midpoint of 90° or 1500µs. With the position the servo is on our robot the midpoint is shifted to 82° or 1411µs. The old midpoint was 66° as the servo's code was redesigned in some changes. The formula to find the Pulse Width µs from degrees is listed below. We used this formula in our calculations for the midpoint where theta is the midpoint degrees we were using. I.e 90°

$$\text{Pulse Width } (\mu\text{s}) = \text{Min Pulse} + \left(\theta \times \frac{\text{Max Pulse} - \text{Min Pulse}}{\theta_{\text{max}}} \right)$$

Figure 3. A screenshot depicting the formula to find Pulse Width µs from degrees.

Below is a table of all the conversions we did which includes our midpoints, as well as the real and theoretical max left and right angles.

Angle (θ)	Pulse Width (500–2500 μ s Standard)	Pulse Width (1000–2000 μ s Alternate)
0° (Max left theoretical)	500 μ s	1000 μ s
42° (Max left real)	967 μ s	1233 μ s
66° (Old Center)	1233 μ s	1367 μ s
82° (New Center)	1411 μ s	1456 μ s
90° (Theoretical midpoint)	1500 μ s	1500 μ s
122° (Max right real)	1856 μ s	1678 μ s
180° (Max right theoretical)	2500 μ s	2000 μ s

Below is a table of our calibrations:

Test	Result	Final value
Steering Centre	Wheels Physically Straight	82° or 1411 μ s
Maximum Left	Safe steering limit	42° or 967 μ s
Maximum Right	Safe steering limit	122° or 1856 μ s
Driving Speed	Stable navigation	1622 μ s

8. Obstacle Avoidance Strategy

The robot uses the Raspberry Pi Camera for detection and recognition of the obstacle and its location within the course. Based on the Raspberry Pi processing of the camera's image, the decision of which way to move in order to avoid the obstacle is made.

1. Detection

The camera takes continuous pictures of the circuit. Using computer vision, obstacles can be detected in the robot's path.

2. Classification

It is decided where the obstacle is located with respect to the robot – whether left, centre or right. In addition, using the camera, course colours are identified.

3. Decision

The robot determines which direction it should move around the obstacle in.

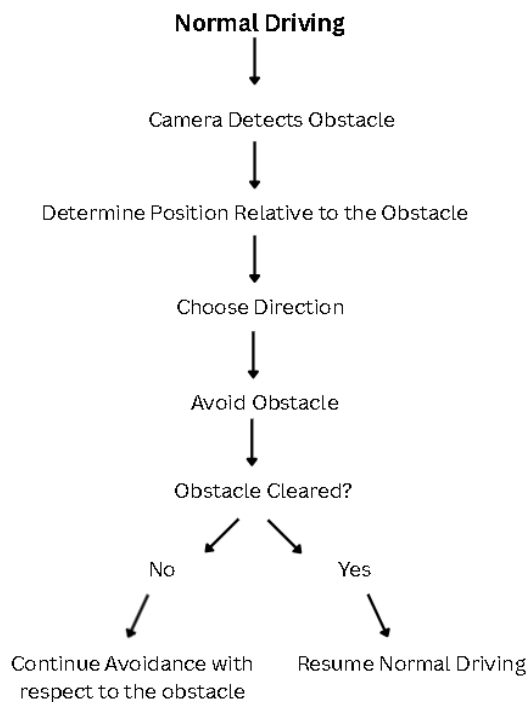
4. Avoidance

While making its way around the obstacle, the robot constantly processes camera pictures.

5. Recovery

Once the obstacle is bypassed, the robot returns to its normal driving path and continues to follow the navigation algorithm.

Flow Chart



9. Parking Strategy

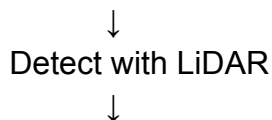
Parking was considered in the design process but was less of a priority when compared to the main driving and obstacle-avoidance strategy. The parking area is detected using the D500 LiDAR and also helps with determining the position of the robot when parking.

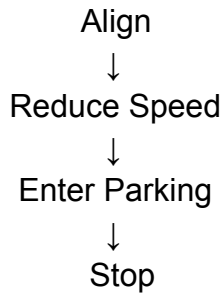
Parking Procedure:

1. Approach – The robot approaches the parking area slowly.
2. Detection – The D500 LiDAR detects the area and assists in identifying the parking spot.
3. Alignment – The robot aligns itself to the parking area.
4. Slow Down – The robot reduces its speed.
5. Park – The robot follows the steering procedure to park in the parking area.

Parking Flowchart

Approach Parking Area





10. Testing & Experiments

Experimentation was critical in designing the robot. Repeated tests were carried out by our team in order to refine the performance of the robot in terms of steering, wall-following, vision processing, avoiding obstacles, making turns, and movement speed.

The main idea behind each experimentation process was modifying a single variable while monitoring its effects on the robot. Based on the outcome, the modification would either be accepted, changed, or undone.

10.1 Experimentation Log

Test	Variable	Result	Conclusion
1	Steering angle / servo midpoint	The initial midpoint value for the servos was set at 66° . However, after testing the robot's turning ability and its behavior, the new midpoint was set to 82° .	82° provided a more reliable and consistent steering centre, so it was used in the final system.
2	Wall distance / wall-following measurement	Many experiments were conducted on the walls of the left and right sides of the camera to understand the impact of the robot's position on the wall areas.	Measuring the ratio between the left and right wall lengths helped in deciding whether the robot had moved too much to either direction.
3	PID Kp and Kd values	Various values of P and D gains have been tried out for finding an optimum	Using P and D control provided sufficient steering correction while

		point between the two. The values obtained were Kp = 30 and Kd = -2.5.	avoiding unnecessary integral accumulation.
4	Camera colour space and thresholding	Different types of thresholding were used, such as RGB, HSV, and CIELAB/LAB . The thresholds were repeatedly modified to optimize wall and obstacle detection at various lightings.	LAB space was chosen to detect walls since it offered a better distinction between the darker walls and their surroundings. The use of HSV space was maintained to detect the pillars with colors.
5	Driving speed	The recommended motor speed was tested first, followed by additional testing using estimated motor command values between approximately 1500–1750 μs .	1622 μs was selected because it provided a good balance between driving speed and the robot's ability to make reliable steering corrections.
6	Camera ROI size and placement	The camera regions of interest (ROIs) were continuously modified during development. Modifications of the chassis and even slight modifications of the camera placement necessitated re-calibration of the ROIs.	The chosen ROIs offered relevant wall information without having too much image processing involved. ROI positioning is a part of the calibration of the camera itself.
7	Corner-turn angle	Different steering angles were tested for left and right corner turns. The final values were 42° for left turns and 122° for right turns .	Fixed steering angles provided a simple and repeatable way to handle corners without requiring the gyro-based turning system.

8	Obstacle detection	Different color threshold values and contour detection filters were experimented with to distinguish the red and green columns without any false detections.	Combining color threshold with checking for contour size, aspect ratio, position and solidity helped achieve better and more reliable obstacle detection.
9	Pillar avoidance steering	The target positions for the detected pillars were tuned separately. The final target positions were x = 90 for red and x = 550 for green .	The distinct targets enabled the robot to navigate towards the right direction while avoiding each pillar.

10.2 Steering Angles

Objective:

To find the steering angles at which the robot can take successful left and right turns without oversteering.

Hypothesis:

As the steering angles are increased, the robot should be able to take sharper turns, but it may overshoot the desired route.

Method:

Testing with various servo angles while the robot is moving through corners and measuring how well the robot stays on the desired track.

Results:

Many tests were done, and the steering angles were fine-tuned till the robot was successfully able to take the turns.

Conclusion:

Using fixed steering angles worked better than varying the steering angle while taking the turn.

Action:

The current code uses:

- **Left turn: 42°**
- **Right turn: 122°**
- **Centre: 82°**

10.3 Wall-Follow Distance

Objective:

Finding out what is the most reliable distance to keep from the wall when following it.

Hypothesis:

If the robot follows the wall keeping its distance, its movement will be more smooth and there will be no need for steering corrections.

Method:

Test the robot on the track by moving it at various distances from the wall.

Results:

To check how far the robot was from the wall, the difference between the areas visible from the left and right parts of the camera was found out.

Conclusion:

Comparison of the two camera areas proved to be a good way to adjust the robot's distance to the wall.

Action:

Now the robot computes the error according to the left and right wall areas.

10.4 PID Kp

Objective:

Find out what proportional gain is appropriate for following walls.

Hypothesis:

Raising Kp would make the robot react more to error, but high Kp causes oscillation.

Method:

Change the Kp while the robot moves on straight parts and around corners.

Results:

Low Kp values led to slow correction, and high Kp values gave a strong reaction from the steering.

Conclusion:

A moderate Kp value struck a good balance.

Action:

The current value is: **30**

10.5 PID Kd

Objective:

To see whether derivative control would improve the stability of wall following.

Hypothesis:

Derivative control will dampen rapid movements in the steering.

Method:

Various Kd values were tested through the observation of the robot's movements.

Results:

Various values were tested to see the amount of damping needed.

Conclusion:

A little amount of derivative control was needed to dampen steering changes.

Action:

The current value is: **-2.5**

10.6 Camera Threshold

Objective:

Determine robust colour thresholds to detect the course walls and coloured pillars.

Hypothesis:

A change in the colour thresholds would help detect the object and minimize false detection due to illumination.

Method:

Apply different LAB and HSV thresholds in different illuminations.

Results:

Detection of dark walls using LAB colour space and red and green pillars using HSV colour space was successful through the camera.

Conclusion:

The use of different colour spaces to perform different operations was successful.

Action:

The current code has used LAB to detect the walls and HSV to detect the red and green pillars.

10.7 Driving Speed

Objective:

Find the driving speed that will make the robot finish the course fast while still being able to control it effectively.

Hypothesis:

Higher speeds will lead to reduced run time; however, controlling and avoiding obstacles will become increasingly difficult.

Method:

Try different values for the motors on straight paths, corners, and obstacle paths.

Results:

Increased speed made it harder to control the path and avoid obstacles.

Conclusion:

Moderate speed gave a better combination of speed and reliability.

Action:

Current normal driving speed is: **1622**

10.8 Pillar Detection

Objective:

Enhance the robot's performance in identifying coloured pillars

Hypothesis:

Applying size, height, aspect ratio, and solidity filters to the detected contours will improve pillar detection accuracy.

Method:

Apply the filter settings to the camera for various shapes and sizes.

Results:

The detection process discards any object that is small, too short, too wide, or not regular.

Conclusion:

Using a combination of geometric filters helps to increase the accuracy of pillar detection.

Action:

The existing program includes filters for the minimum area, height, aspect ratio, and width.

10.9 Avoiding the Pillar

Objective:

To see if the robot can turn away from a detected red or green pillar.

Hypothesis

It will be possible for the robot to adjust its steering by determining the position of the pillar horizontally in the camera image.

Method:

Place the red and green pillars in different positions and record their responses.

Results:

The robot finds the difference between the center of the pillar that is detected and the target position.

Conclusion:

It is possible for the robot to adjust its steering according to the position of the pillar.

Action:

The current code uses different target positions:

RED_TARGET_CX = 90

GREEN_TARGET_CX = 550

10.10 Testing Process

Testing followed an iterative process:

1. Design
2. Build / Program
3. Test
4. Record Results
5. Identify Problem
6. Change One Variable
7. Retest
8. Keep or Reject Change

This process allowed the team to make changes based on measured performance rather than assumptions.

Testing Philosophy

The main goal of testing was reliability before speed. A change was only considered an improvement if it made the robot more consistent without creating new problems anywhere else on the course.

11. Design Iterations

The robot was developed through several iterations. Each major change was made in response to problems we found during testing.

11.1 Iteration Timeline

1. Initial Prototype

2. Steering Testing
3. Servo midpoint changed: $66^{\circ} \rightarrow 82^{\circ}$
4. Control System Tuning
5. P and D values adjusted; I term removed
6. Vision Testing
7. LAB selected for wall detection
8. Camera / Chassis Adjustments
9. ROIs recalibrated
10. Speed and Obstacle Testing
11. Final Robot

11.2 Major Design Iterations

Iteration 1 – Steering Calibration

Date: 2026-09-01

- **What changed:** Changed the steering midpoint from 66° to 82° .
- **Why:** The initial midpoint failed to provide reliable straight line steering.
- **Test result:** The robot drove straighter.
- **Pros:** Improved straight line driving and steering.
- **Cons:** Needed to calibrate other steering values around the new midpoint.

Iteration 2 – Control System

Date: 2026-02-08-2026-07-16

- **What changed:** Tried several P and D values and received $K_p = 30$, $K_d = -2.5$. I term was not used.
- **Why:** Need to tune the control system.
- **Test result:** P and D control was enough to correct the flat course.
- **Pros:** Better control when following walls.
- **Cons:** Much time spent on testing and tuning.

Iteration 3 – Vision System

Date: 2026-05-10

- **What changed:** Tested RGB, HSV and LAB. Lab colour space chosen for wall detection and HSV for coloured pillars.
- **Why:** Different colour spaces gave different results.
- **Test result:** LAB colour space worked better for wall detection in our environment.

- **Pros:** Better differentiation between walls and the background.
- **Cons:** Miss out on the ease of use with RGB

Iteration 4 - Camera and ROI placement

Date: 2026-03-31, 2026-04-12, 2026-04-18, 2026-05-03, 2026-06-21, 2026-06-25

- **What changed:** The ROIs of the camera were changed multiple times.
- **Why:** Changing the chassis or camera placement affected its field of vision.
- **Test result:** New ROIs helped improve wall detection.
- **Pro:** Improved focus of image processing.
- **Con:** Mechanical changes required new software adjustments.

Iteration 5 - Speed of driving

Date: 2026-07-25

What changed: Motor values from approximately 1500-1750 μ s were tested.

Why: There needed to be a compromise between speed and proper steering & image processing.

Test result: 1622 μ s was the value providing the best results.

Pro: Increased reliability without reducing speed.

Con: Higher speed may decrease reaction time.

Date uses a year/month/day format

11.3 Final Robot Design

The final robot incorporates all the improvements found during testing such as 82° steering center point, P & D tuned control, camera colour processing, calibrated ROIs, and a BLDC motor PWM of 1622 μ s.

12. Problems & Solutions

Problem	Investigation	Cause	Solution and Result
---------	---------------	-------	---------------------

Unexpected startup movement	Checked the Arduino reset, startup delay, and initial motor commands.	The Raspberry Pi and Arduino did not always initialize at the same time.	Added a startup delay and controlled motor initialization, making startup more predictable.
Unreliable camera detection	Tested RGB, HSV, LAB, and different threshold values.	Lighting and colour variation affected detection.	Used LAB for wall detection and HSV for coloured pillars, improving detection reliability.
Camera ROIs became inaccurate	Checked the camera view after chassis or camera-position changes.	The ROIs depended on the camera's physical position and angle.	Recalibrated the ROIs after mechanical changes and kept the final camera position consistent.
Unreliable obstacle detection	Tested colour thresholds and contour filtering.	Colour alone could detect noise or invalid objects.	Added filters for area, size, position, aspect ratio, and solidity, making pillar detection more selective.
Driving speed affected control	Tested motor commands between approximately 1500–1750 μ s.	Higher speeds reduced the time available for camera processing and steering corrections.	Selected 1622 μs , providing a better balance between speed and reliable control.

13. Performance Analysis

The final robot was assessed through the criteria of speed, accuracy, reliability, and power consumption. These criteria helped determine if the changes made to the design were effective.

13.1 Performance Summary

Category	Measurement	Result
Speed	Maximum speed	32.23 m/s
Speed	Average speed	4.1 – 6.8 m/s
Accuracy	Steering error	~15%
Accuracy	Wall distance error	~8/10
Accuracy	Obstacle detection accuracy	~8/10
Reliability	Successful runs	9/10
Reliability	Failed runs	1/10
Reliability	Collision rate	1/10
Power	Average current	Approximately 3.5 A
Power	Peak current	Approximately 15 A
Power	Battery runtime	Approximately 40–60 minutes

13.2 Performance Comparison

Robot Version	Best Time	Successful Runs	Main Improvement
Initial Prototype	2:46	2/10	Baseline performance
Steering-Calibrated	2:12	5/10	Improved steering consistency
Control and vision	1:57	7/10	More stable wall following and detection
Final	1:37	9/10	Combined mechanical and software improvements

open challenge times

13.3 Analysis

The final robot's performance should be compared with the previous robots to see whether design changes improved its reliability and controllability. The key criteria are accurate steering, obstacle detection, number of successful runs, and completion time.

The robot's power system consists of a battery with an average current of about 3.5 A, peak current of about 15 A, and battery operation time of 40–60 minutes.

13.4 Evidence

The performance evidence was collected and will be collected moving forward from:

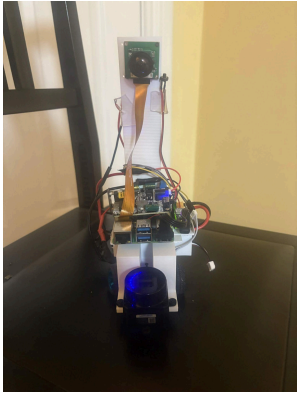
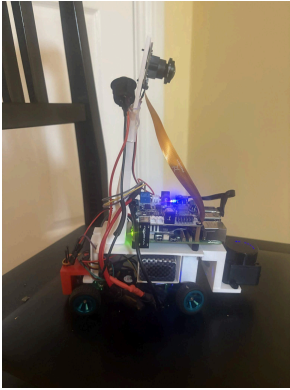
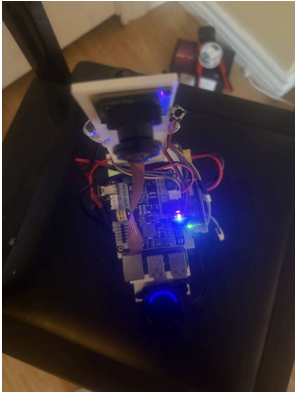
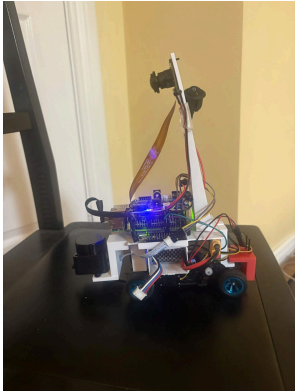
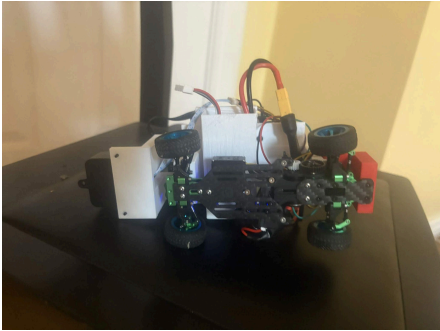
- Timed driving trials.
- Successful and failed run records.
- Collision records.
- Camera detection results.
- Steering and wall-following observations.
- Battery and current measurements.
- Competition results and performance videos.

14. Final Robot

The final robot uses all the changes made to the mechanical, electrical, and programming aspects throughout the process of designing. The completed design will be able to navigate through the course autonomously, detect walls, obstacles, and manoeuvre as required in the competition.

14.1 Final Robot Views

The photos below show the completed robot from all the major views possible. Components and important features are labelled for identification.

View	Image	View	Image
Front		Right	
Back		Top	
Left		Bottom	

14.2 Final Specifications

Specification	Final Value
Computer	Raspberry Pi 5
Controller	Arduino Nano
LiDAR	D500
Camera	Raspberry Pi Camera
Motor	Furitek Micro Komodo 1212
ESC	Furitek Lizard Pro
Steering	Servo motor
Battery	11.1 V 2200 mAh 3S LiPo
Average power	~38 W
Average current	~3.5 A
Peak current	~15 A
Weight	567g
Dimensions	Length: 21cm, Width: 10cm, Height: 23.5cm
Maximum speed	32.23 m/s

14.3 Final System

The Raspberry Pi 5 conducts the primary computer vision and decision-making functions. The Arduino Nano executes the lower-level operations such as steering and control of the motors. The camera gives the vision data for wall and obstacles detection, and the D500 LiDAR is used for the parking system.

This is the final robot which reflects the outcome of the work carried out during the mechanical, electrical, and programming developments described in the engineering journal.

15. Reproducibility

The robot was documented such that another team would be able to replicate the design based on the documents contained in this repository. All the major mechanical, electrical, and software systems have been documented along with their respective configurations and calibrations.

15.1 Parts

Component	Manufacturer	Model
Computer	Raspberry Pi	Raspberry Pi 5
Microcontroller	Arduino	Arduino Nano
Camera	Raspberry Pi	Raspberry Pi Camera
LiDAR	LDROBOT	D500
Motor	Furitek	Micro Komodo 1212
ESC	Furitek	Lizard Pro
Servo	HiTEC	HS-5055MG Metal Gear Digital Micro Servo
Battery	Zeee	3S 11.1 V 2200 mAh LiPo

Complete component specifications and additional parts are documented in the repository.

15.2 Mechanical Construction

The mechanical design is documented using:

- CAD files
- Dimensioned drawings
- 3D-print files
- Component measurements
- Assembly photographs

All the CAD files, 3D-print files, and mechanical drawings can be found on our github.

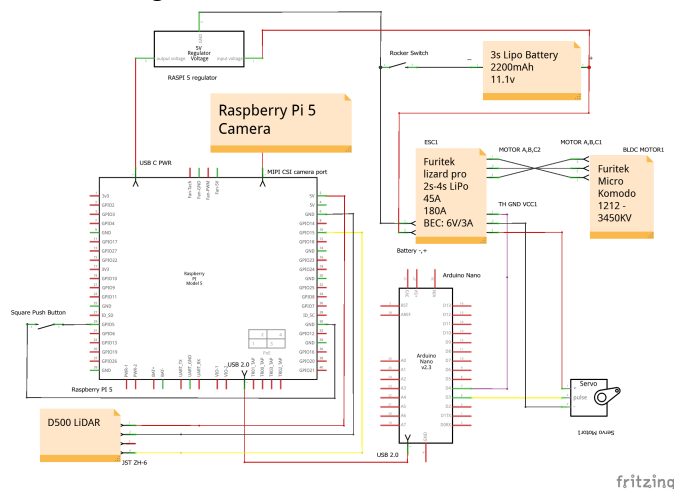
15.3 Electronics

The electrical system is documented using a complete wiring diagram and component connections.

Documentation includes:

- Wiring diagram
- Raspberry Pi connections
- Arduino pinout
- Sensor connections
- Motor and ESC connections
- Power distribution
- Voltage regulation

Circuit diagram:



15.4 Software

The complete software is available in the repository.

Requirements

- Raspberry Pi 5
- Python 3
- OpenCV
- NumPy
- Picamera2
- Serial communication libraries
- GPIO libraries

Configuration

The software requires configuration of:

- Camera settings and ROIs
- LAB and HSV thresholds
- Steering centre
- Motor values
- PID parameters
- Obstacle-detection parameters

The final configuration values are documented alongside the source code which can be found on our github as well, in the designated “Code” folder.

15.5 Calibration

Before operating the robot, the following systems must be calibrated:

1. **Steering:** Set the servo centre and verify that the wheels point straight.
2. **Motor:** Determine the neutral value and gradually increase the motor command until the desired driving speed is reached.
3. **Camera:** Position the camera consistently and adjust the ROIs and colour thresholds.
4. **LiDAR:** Verify distance readings at known distances and confirm the sensor's orientation.
5. **Sensors:** Check that all sensor readings are received correctly by the Raspberry Pi and Arduino.

15.6 Assembly Order

The recommended assembly order is:

1. Assemble the chassis.
2. Install the motor, wheels, and steering system.
3. Mount the Raspberry Pi, Arduino, ESC, and battery.
4. Install the camera and LiDAR.
5. Complete the electrical wiring and voltage regulation.
6. Upload and configure the software.
7. Calibrate the steering and motor.
8. Calibrate the camera and sensors.
9. Test each subsystem individually.
10. Perform complete autonomous testing.

15.7 Reproducibility Resources

The repository contains the files required to recreate the robot, including:

- **CAD and mechanical files**
- **Electrical schematic**
- **Source code**
- **3D-print files**
- **Component specifications**
- **Calibration values**
- **Assembly photographs**
- **Testing and configuration information**

This is the link to our GitHub: <https://github.com/JBH1229/WRO-2026-FE>

15.8 Calibration Values Table:

Setting	Final Value
Steering centre	82°
Motor neutral	1500 μ s
Driving speed	1622 μ s
Kp	30
Kd	-2.5
Camera resolution	640 $\times$ 480
Red target X	90
Green target X	550

16. GitHub / Code

The GitHub repo contains the source code for the robot, the engineering documentations, the electrical schematics, and other files. This repository allows for access to the development history as well as the final product.

16.1 Repository Contents

The main sections of the repository contain:

Section	Contents
Engineering Journal	Documentation of the robot's development and testing
Software	Python and Arduino source code
Electrical	Circuit diagrams, wiring, and electrical documentation
Mechanical	CAD files, designs, and mechanical documentation
Images	Robot photographs, diagrams, and testing images

16.2 Software

The robot's software is divided between the **Raspberry Pi 5** and **Arduino Nano**.

Raspberry Pi

The Raspberry Pi runs the main program and is responsible for:

- Camera Processing
- Wall detection
- Obstacle detection
- Navigation decisions
- LiDAR communication
- IMU monitoring
- Sending motor and steering commands

Arduino Nano

The Arduino handles the lower-level hardware control, including:

- Steering servo commands
- Motor commands
- IMU communication
- Communication with the Raspberry Pi

16.3 How the Software Works

The main autonomous program can be shown through a state-machine diagram structure.

1. Wall Following
2. Corner Detected
3. Corner Turn
4. Wall Following
5. Pillar Detected
6. Pillar Avoidance
7. Recovery
8. Wall Following

The Raspberry Pi constantly processes information from the camera and decides which main state the robot is in. After that, the steering and motor control commands are sent to the Arduino.

16.4 Installation and Running

The software requires a Raspberry Pi 5 with the required Python libraries and connected hardware.

Basic setup:

1. Clone or download the repository.
2. Install the required Python dependencies.
3. Connect the Raspberry Pi to the Arduino and sensors.
4. Verify the correct serial ports.
5. Check the camera and sensor connections.
6. Apply the documented calibration values.
7. Run the main autonomous program.

Make sure to calibrate the steering, motor speed, k_p & k_d , and the red & green pillars before running.

16.6 Version History

The commit history on GitHub records the evolution of the robot and also allows the previous versions of code and documentation to be viewed. This provides evidence towards the evolution of the software through the entire engineering process.

Therefore, the repository can act as both a code archive and development record, as it shows how the final robot was produced throughout all the variations in software and hardware.

17. Engineering Goals

At the start of the project, we defined engineering objectives to assess the performance and reliability of the robot. These engineering objectives were used to determine if our final design lived up to our initial expectations.

Goal	Target	Final	Achieved?
Speed	<2:30	1:37	✓
Wall follow & pillar error	>80%	~80%	✓
Runtime	>2h	~40-60m	✗
Successful runs	>80%	~90%	✓

18. Future Improvements

While the final robot successfully combines autonomous drive, wall following, obstacle avoidance, and sensing navigation together effectively, there are still some improvements that can be made to the robot in a future iteration. Each improvement is clear-cut and will be testable based on performance measurements.

18.1 Mechanical

- **Decrease weight:** Modify the chassis design and strip out unnecessary components to decrease the total weight of the robot.
- **Enhance steering geometry:** Optimize the steering mechanism design to decrease steering slack and increase consistency between the servo and wheel positions.
- **Improve chassis stiffness:** Make stronger sections of the chassis that flex when the robot is moving to ensure the camera and steering geometry stay consistent.
- **Enhance wheel traction:** Experiment with different wheel designs to come up with one that increases traction without decreasing roll resistance.

18.2 Electronics

- **Increase power regulation efficiency:** Use an improved voltage regulator for minimizing variations in voltage and ensuring better power supply to the Raspberry Pi and sensors.
- **Increase cable routing effectiveness:** Increase the quality of cable routing and soldering to minimize chances of improper cable connections.
- **Minimize electrical noise:** Organize power supply of the motor system and other cables in such a way as to minimize interference of electrical nature.
- **Improve sensors mounting reliability:** Mount sensors in such a way that they don't move during collisions or trials.

18.3 Sensors

- **Enhance camera calibration:** Implement a more systematic procedure of calibrating cameras such that thresholds and regions of interest are consistent regardless of varying lighting situations.
- **Improve LiDAR filtering:** Incorporate further filtering in order to eliminate unreliable or noisy LiDAR data before they are used by the robot.
- **Implement improved sensor fusion:** Combine information from the camera, LiDAR, and IMU whenever appropriate to aid navigation.

18.4 Software

- **Obstacle detection improvement:** Refine colour thresholding and contour filtering techniques further in order to avoid incorrect detections and provide consistent obstacle detections.
- **Control algorithm improvement:** Try out some more advanced control strategies in order to decrease the robot's oscillations while driving and provide more consistent distance from the wall.
- **Latency reduction:** Improve image processing and sensor processing in order to decrease robot's response time to environment changes.
- **Improvement of recovery state:** Implement improvements to the recovery state in order to let the robot return to regular wall following mode after recovering from an obstacle or turning a corner.

18.5 Strategy

- **Enhancement of Obstacle decisions:** Look at more methods of obstacle avoidance to see what approach would yield the fastest route.
- **Optimization of Driving Path:** Modify the target wall position and steer settings to come up with a faster path but ensure reliability of obstacle detection.
- **Cornering Enhancement:** Experiment with various entry speed and angles of the corner to decrease time used on corners.
- **Parking enhancement:** Further optimize the parking approach using the LiDAR.

18.6 Testing Future Improvements

Future improvements will be tested by comparing the modified robot and the existing one under the same conditions. With respect to each of the changes made, several trials will be done, and various measurable parameters such as lap time, lateral error/wall error, success rate, and collisions will be recorded.

For instance, in testing a better control algorithm, both the present control system and the enhanced one will be run on the same track for ~20 runs. Comparing the two systems will be based on various parameters such as the average wall error, maximum steering error, lap time, and the number of successful runs. The new control algorithm in this example would only be considered an improvement if it produced enough gains without reducing reliability or negatively affecting other parts of the robot

This process would allow future changes to be based on experimental evidence rather than assumptions.

19. Links

Link to Github: <https://github.com/JBH1229/WRO-2026-FE>

Link to Open Challenge video: <https://youtu.be/NveL-KeYepw>

Link to Obstacle Challenge video: <https://youtu.be/7QU7L7XVNVw>